



Combining traditional machine learning and anomaly detection for several imbalanced Android malware dataset's classification

Yiwei Gan
Dalhousie University, Halifax, NS
B3H 4R2, Canada

Qian Han
Dartmouth College, Hanover, NH,
United States

Yumeng Gao
Institute of Infocomm Research,
Agency for Science, Technology and
Research (A*STAR), Singapore

ABSTRACT

As the number of mobile devices has exploded in recent years, so has the amount of advanced Android malware. Among the popular Android malware on the market today, click fraud malware, adware, banking Trojans, spyware, etc. are usually disguised and hidden in the heap of good Android applications. These advanced malwares lurk in the third-party application market trusted by users, and potentially endanger the security of the user's smart device causing privacy or economic loss. Therefore, this paper leverages and combines traditional machine learning and anomaly detection methods to detect specific classes of Android malware in three highly imbalanced Android datasets (entertainment + social app vs. click fraud malware + adware; financial services app vs. banking trojan; communication app vs. spyware). The experiment results show that our proposed combined methods have great performance on the three sub-datasets, achieving the average f1-score over 0.98 on three imbalanced datasets, which performs better than the traditional machine learning algorithm used alone. In addition, we use the combined methods to analyze the correlation between the top features of the dataset, and provide interpretable insights for other researchers focusing on Android malware classification in the coming future.

CCS CONCEPTS

- Information systems; • Information systems applications; • Mobile information processing systems;

KEYWORDS

Android malware, machine learning, data mining, static analysis, mobile security

ACM Reference Format:

Yiwei Gan, Qian Han, and Yumeng Gao. 2022. Combining traditional machine learning and anomaly detection for several imbalanced Android malware dataset's classification. In *2022 7th International Conference on Machine Learning Technologies (ICMLT) (ICMLT 2022)*, March 11–13, 2022, Rome, Italy. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3529399.3529412>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICMLT 2022, March 11–13, 2022, Rome, Italy

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9574-8/22/03...\$15.00

<https://doi.org/10.1145/3529399.3529412>

1 INTRODUCTION

In recent years, as the technological era shifts from PC to mobile, the use of smart devices has grown significantly. According to the latest forecast data, there will be more than 4 billion smart device users in the world by 2023 [1]. At the same time, Android has firmly occupied the share of the major mobile operating systems, with a market share of nearly 72%, far exceeding the market share of the second-largest mobile manufacturer, iOS, which is close to 27%. The worldwide popularity of Android smart devices has made them the primary target of cybercriminals. Some cybercriminals endanger users' security by creating malware and viruses, causing serious damage to users' property and privacy. Some existing research results have used machine learning methods to study malware attacks [2–4], and they have achieved relatively good experimental results. The advantage of using machine learning for malware detection is that we only need to utilize a relatively small number of positive and negative samples to build a high-quality dataset, and then use the state-of-the-art machine learning classifier for training, and then we can achieve large-scale high-precision sample prediction, which saves the efforts of manually constructing rules and signatures comparing to the traditional way.

Most of the existing research [5–7] on Android malware detection focuses on a broader binary classification, such as using supervised-learning classifiers to predict whether an Android app is a malware or not directly. The sample datasets are usually built with general good apps and suspicious apps, but this part of the research usually does not consider that the difficulty of obtaining suspicious or bad samples is much higher than that of good samples in reality, which may cause the performance of trained classifiers to decline in the case of imbalanced datasets. Another part of the research [8, 9] focuses on clustering similar Android malware samples using unsupervised learning, but in this case, it is generally difficult to obtain good interpretability of the most important factors that distinguish good and bad samples. Sometimes, the cluster may have a relatively high probability of clustering bad apps with similar functions into a large category of good apps (e.g. spyware is mistakenly clustered into communication apps), so that malware can escape the malware detection.

Therefore, in this paper, we propose a method that combines traditional machine learning and anomaly detection to achieve high-precision detection of three imbalanced datasets (Entertainment + Social apps vs. Click Fraud + Adware; Finance apps vs. Banking Trojans; Communication apps vs. Spyware) in the field of Android malware classification. The proposed method solves the shortcomings of the existing research proposed above, such as 1) it is relatively difficult to obtain malware samples compared to goodware samples; 2) using a single classifier in malware detection

may cause the regression of model performance; 3) malware with similar functions with goodware is easy to hide between goodware to escape detection when both of them share similar features. All our experimental results show our proposed method achieved the best performance on the built datasets over using each classifier alone.

The main contributions of this paper are as follows:

- The proposed algorithm is the first to combine traditional machine learning classification and anomaly detection algorithms to the recent imbalanced 3 sub-divided Android malware dataset to the best of our knowledge. The experiment results show that the proposed methods can achieve an F1-score over 0.98 on all three datasets, which perform better than traditional machine learning algorithms such as random forest or decision trees.
- This paper analyzes the importance of the top 30 features that can distinguish a good app from specific malware. At the same time, we simply analyze and draw the heatmap of the correlation coefficient of these features, and perform qualitative and quantitative analysis.

The rest of the paper is organized as follows: Section 2 briefly describes the literature review of Android malware static and dynamic analysis techniques and machine-learning-based detection techniques. Section 3 presents the evaluation of the experiment pipeline of the paper. Section 4 discusses the related experiments and results, including the imbalanced used datasets, evaluation metrics, classification results, and top feature analysis. Finally, Section 5 draws conclusions and future work of our paper.

2 LITERATURE REVIEW

2.1 Static analysis and Dynamic analysis of Android Malware

There are generally three ways to analyze apk, namely static analysis, dynamic analysis, and hybrid analysis. Static analysis generally decompiles/reverse-engineers the apk, extracts and analyzes the bytecode and source code of the apk. Usually, the static analysis does not actually use the sandbox to run and compile the apk in real-time to extract its dynamic behavior. Static analysis usually includes analyzing apk's API call graph, Control Flow Graph, Smali code, etc [10, 11]. Comparatively, the dynamic analysis generally uses a sandbox isolated in a secure environment to simulate the actual operating environment of apk, and records apk runtime logs, from which to analyze its network upload and download traffic, binary files read and write operation, and other suspicious behaviors, among which URL, IP address, system resource scheduling, and user's sensitive information are the focus of the malware itself [12, 13]. The hybrid analysis is a combination of both static analysis and dynamic analysis [14].

2.2 Machine Learning-Based Android Malware Detection

There have been many related studies discussing Android malware detection technology based on machine learning. For example, SigPID [15] mainly uses the Manifest file to extract 22 important relevant permission features for malware classification. They also

applied Naive Bayes, Decision Tree, and SVM on their feature set, and the model accuracy reached 90% with the use of SVM. Other examples [16, 17] use the Drebin [18] as their dataset. These papers analyze and extract the bytecode, API call, and operation code of the apk, and used a variety of traditional machine learning classifiers. All of them found that with the use of the random forest, it can achieve the best classification results with an accuracy of more than 94%.

To the best of our knowledge, we are the first work combining anomaly detection with traditional machine learning classifiers and applying the combined classifiers to three specific highly imbalanced datasets to differentiate from the above studies. Our study achieves very high classification results and we also conduct detailed top feature analysis. Therefore, our work in using machine learning to classify Android malware is unique and innovative.

3 PIPELINE

The whole pipeline of imbalanced Android malware classification is shown in Figure 1. First, we collected all recent Android APK samples as a dataset, and then we used AndroPyTool [19] to extract static features for machine learning analysis. We then categorize all the apps (goodware & malware) according to the Google Play Store top free app classification and malware based on the official category by NJCCIC [20]. Three imbalanced datasets (e.g. Entertainment + Social vs Click Fraud + Adware) are obtained by categorization based on the realistic relevance of each app and malware category. Next, we put three different datasets into four state-of-the-art supervised learning classifiers and two anomaly detection classifiers and use the late fusion method to compute the prediction results. In addition, we also used the Random Forest model to extract top features and repeated the above training process to distinguish whether a small number of feature vectors would be extra helpful for the classification of Android malware and Android goodware.

4 EXPERIMENT RESULTS

4.1 Datasets

In our experiment, we collected 105 types of malware with 6473 samples in total: 325 Click Fraud, 656 Adware, 1172 Banking Trojans, 807 SMS Fraud, 2825 Spyware, 279 Toll Fraud, 570 Backdoors, 307 Hostile downloader, 187 Ransomware, 680 Privilege escalation (rooting), 50 Phishing, and 564 other Trojans. The malware datasets are collected from the 2017 Canadian Institute for Cybersecurity dataset named CICAndMal2017 [21]. Also, we collected 6633 goodware with 114 types of unique apps from the Google play store. The features of these samples are extracted by AndroPyTool [19].

The categorized datasets we collected are as Table 1 shows: 1. Entertainment + Social vs Click Fraud + Adware, 2. Finance vs Banking Trojans, and 3. Communication vs Spyware. The first dataset contains goodware which has 1600 samples with 27 types of apps, and malware which has 1792 samples with 16 types of apps. The second contains goodware which has 895 samples with 4 types of apps, and malware which has 896 samples with 4 types of apps. The third contains 895 goodware samples with 12 unique apps, and malware which has 895 samples with 47 types of apps.

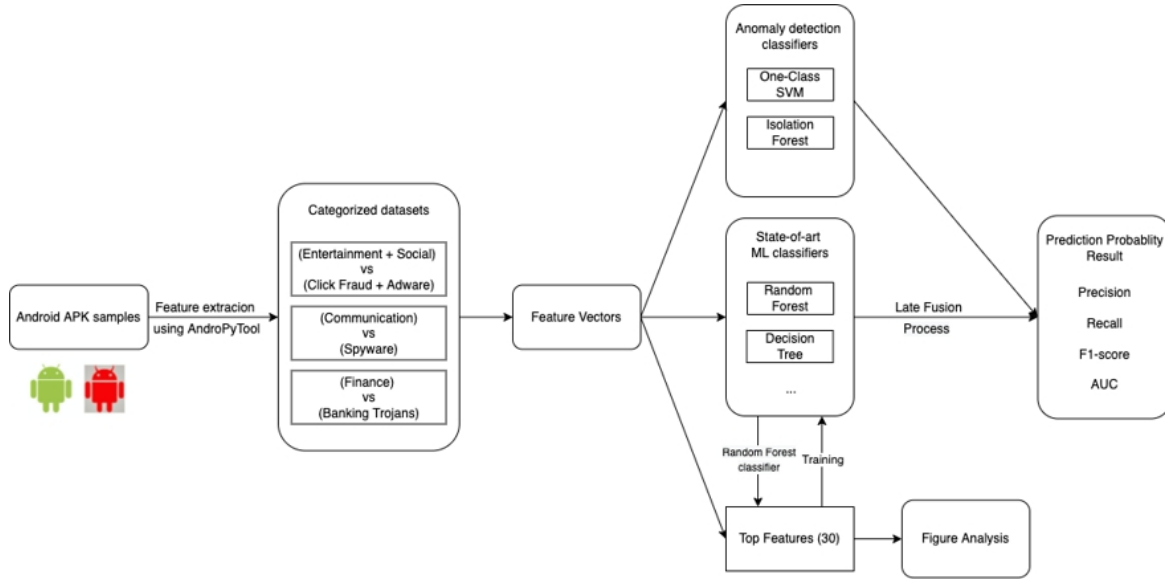


Figure 1: The pipeline of the whole classification process

Table 1: Details of three categorized datasets

Goodware vs Malware	Category	Number of rows	1st Test 9% malware in dataset	2nd Test 20% malware in dataset
Entertainment + Social vs Click Fraud + Adware	Entertainment + Social	1600	1600	1600
	Click Fraud + Adware	1792	176	400
Finance vs Banking Trojans	Finance	895	243	243
	Banking Trojans	896	24	61
Communication vs Spyware	Communication	895	760	760
	Spyware	47	76	190

According to our observation of the relationship between different goodwill and malware at the real-world level, we found some of the following connections, which led to three categorized partitions for their different usage scenarios:

- For Entertainment + Social vs Click Fraud + Adware, most entertainment and social apps are mainly used for reading graphics, watching videos, and playing games. A major part of the profitability of these two types of goodwill may be advertising, when users view and click on unauthenticated links and advertisements, their personal information may be violated, resulting in privacy theft and economic loss. So we think that in this type of usage scenario, these four categorized goodwill and malware can form a data frame.
- For Finance vs Banking Trojans, it is due to the high correlation between usage scenarios and functions.
- For Communication vs Spyware, we generally believe that the information transmitted between people is diversified in the communication apps, and the possibility of users being overturned by spyware is more significant.

After categorizing the three datasets, we set the train-test-split into two cases: 91% goodwill vs 9% malware and 80% goodwill vs 20% malware. Three datasets under categorized are already in an imbalanced state. Thus, when we split them through these two methods, we can get a more imbalanced status which is closer to the industrial status. Moreover, this data pre-processing method can more effectively help to evaluate the performance in a real industrial circumstance.

4.2 Metrics

The measurement indexes of common machine learning models mainly include four parameters: Precision, Recall, F1-Score, and AUC. We also adopt such measures in our research to judge the effect of the model in different situations. TP means True Positive; FP means False Negative; FN means False Negative in the following definition:

- Precision: A measure of the possibility of an actual positive sample among all predicted positive samples. $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$.
- Recall: A measure of the possibility that an actual positive sample is predicted to be a positive sample $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$.
- F1-score: F1-score is a balance measurement between Precision and Recall scores and gives an overall score. $\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$.
- AUC: AUC stands for "the areas under the ROC Curve". AUC provides a comprehensive performance measure for all possible classification thresholds. It measures the entire 2D area under the ROC curve from (0,0) to (1,1). It compares the true positive rate and the false positive rate. Therefore, the larger AUC means the greater difference between true positives and true negatives.

4.3 Classification Results

As the following Table 2 - Table 4 show below, we provide the above four measurement parameters for each of the seven different models. Among them, the first is the result of Late fusion, the second to fifth are the result of traditional machine learning models, and the last two are the results of anomaly detection models. Also in the four charts below, we provide the above four measurement parameters for each of the seven different models.

Among the four traditional machine learning models, Decision Tree and Random Forest have stable and high performances. Decision Tree aims to select a specific number of features from the training set as nodes for derivation. This supervised model with recursion can generate top-down child nodes based on the evaluation criteria. Random Forest is a simple, easiest-to-implement, and low-computational-cost model. Its baseline is Decision Tree while integrates with the Bagging method and random selection method. One-Class SVM belongs to the support vector machine algorithm, but it is an unsupervised learning model. It does not need people to mark the output labels of the training set. Different from traditional classification algorithms, the single classification model can be used in data that only has positive samples been labeled. This is suitable for using this model to quickly determine results when new malware is not yet labeled. Isolation Forest mainly uses the idea of ensemble learning to detect outliers and is currently an important algorithm in outlier detection. The basis of its algorithm is random forest and only needs to input a small number of samples for recursion Tree instead of sampling all the training set in Random Forest. This model can use a randomness method for selecting features and thresholds until only one sample is left. Calculating the average path length of $h(x)$ of each tree is the first step, and then we will compute the anomaly probability score x .

In late fusion, we used 4 models for late fusion, including Decision Tree, Random Forest, One-Class SVM, and Isolation Forest. Late fusion is in the form of the grid search, and their parameter settings are adjusted in a range from 0 to 1 with 0.2 as each step. The input is the prediction probability score which is generated by each selected model. The main reason for choosing 4 models is that among the various models, the combination of the above four models achieves a stable and significantly higher effect. Through

Table 2: Classification results on Communication apps vs Spyware with the imbalanced ratio of 80% goodwill vs 20% Malware using top features

Model	Precision	Recall	F1-Score	AUC
Late Fusion - 2nd Test (Top Features)	0.9940	0.9940	0.9940	0.9688
Random Forest	0.9967	0.9688	0.9822	0.9688
Decision Tree	0.9967	0.9688	0.9822	0.9688
Logistic Regression	0.9810	0.8125	0.8749	0.8125
Naive Bayes classifier	0.7965	0.9359	0.8472	0.9359
One Class SVM	0.0148	0.0417	0.0219	0.4211
Isolation Forest	0.0088	0.0625	0.0154	0.2286

Table 3: Mean classification results on all three imbalanced datasets with the imbalanced ratio of 91% goodwill vs 9% Malware using all features

Model	Precision	Recall	F1-Score	AUC
Late Fusion - 1st Test (All Features)	1	1	1	1
Random Forest	1	1	1	1
Decision Tree	0.9989	0.9896	0.9941	0.9896
Logistic Regression	0.9463	0.8642	0.8659	0.8642
Naive Bayes classifier	0.7670	0.9312	0.8167	0.9312
One Class SVM	0	0	0	0.5000
Isolation Forest	0.0344	0.3100	0.0619	0.5304

Table 4: Mean classification results on all three imbalanced datasets with the imbalanced ratio of 80% goodwill vs 20% Malware using top features

Model	Precision	Recall	F1-Score	AUC
Late Fusion - 2nd Test (Top Features)	0.9961	0.9961	0.9961	0.9853
Random Forest	0.9973	0.9782	0.9874	0.9782
Decision Tree	0.9973	0.9782	0.9874	0.9782
Logistic Regression	0.8860	0.8541	0.8613	0.8541
Naive Bayes classifier	0.7996	0.9172	0.8410	0.9172
One Class SVM	0.0144	0.0366	0.0206	0.4026
Isolation Forest	0.0153	0.1016	0.0266	0.2612

the parameter setting set in late fusion, we observed that not only do the anomaly detection models occupy a high proportion from 0.5332 to 0.6, but also the selected traditional machine learning models have a stable proportion from 0.3999 to 0.4666 as Table 5 shows. Selecting all models for late fusion will not only cause the parameter settings between models to be scattered but also the waste of computation.

The F1-Score in four traditional machine algorithms ranges from 0.8472 to 0.9822, of which Random Forest and Decision Tree perform the best result on mere classification. But the score is significantly low from 0.0154 to 0.0219 while only training the anomaly detection models. In the following tables, "1st Test" means we use all features while the imbalanced ratio is 91% goodwill vs 9% malware, while "2nd Test" means we use the top features in ML models with the imbalanced ratio 80% goodwill vs 20% malware.

4.4 Late fusion result analysis

However, its deep-seated reasons for the weight distribution require further in-depth research. Anomaly detection models may

Table 5: Average classifier parameter weight for late fusion for all 3 imbalanced datasets with the imbalanced ratio of 80% goodwill vs. 20% malware

	Decision Tree	Random Forest	One-Class SVM	Isolation Forest
1st Test - All features	0.1666	0.2333	0.5	0.1
2nd Test - Top Features 80% Good wares vs 20% Malwares	0.1666	0.3	0.3666	0.1666

capture outliers/margins that traditional machine learning models cannot classify. The reason may lie in the distribution of features. The traditional machine learning models generally compute a fuzzy average score from the inputted features. The performance showed significance in both cases (all features and top features), which means selected high-dimensional features are essential to the model. Though One-Class SVM's feature [22] is good at processing high-dimensional features, results from Table 2 to Table 4 do not present well. We would assume that the distribution of features either aggregates together in the hyperplane or is extremely separated. However, since there is a strong correlation between most features in the heatmap, we prefer that the data features are more aggregated. Most of the features in the heatmap as Figure 3 shows present a high-correlation coefficient, which is inconsistent with the result generated by late fusion which has a large proportion for anomaly detection models. However, combined with Table 5, it can be observed that the parameter setting in late fusion containing both traditional machine learning models and the anomaly detection models, shows that both are relatively equally important.

After the above experiments, we can only partially prove that the feature of the dataset is high dimensional, but cannot prove whether the feature is concentrated or scattered in the hyperplane. However, due to the characteristics of One-Class SVM itself, excessive concentration and dispersion cannot allow the model to accurately find outliers. Thus, we know that high-dimensional data

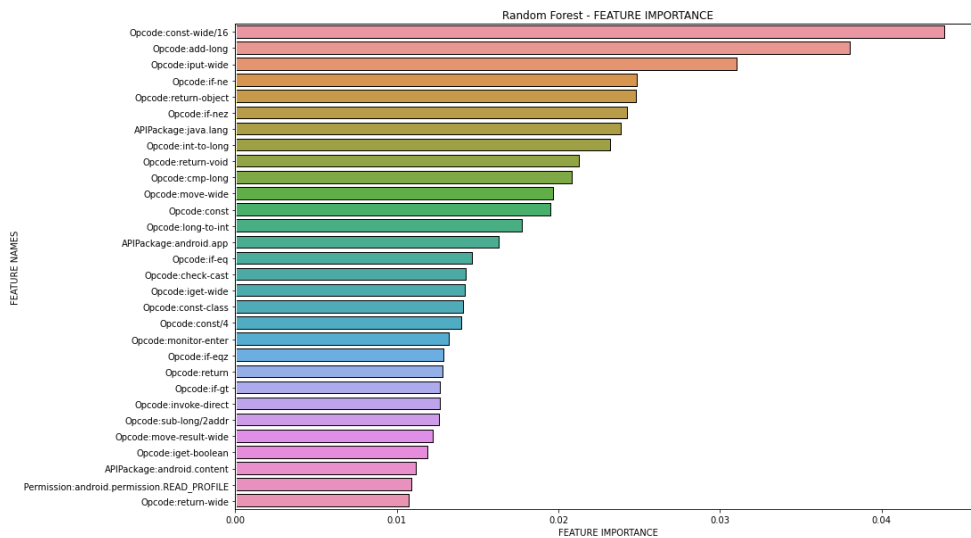
will have different performances in different models. And we lack proper methods to disassemble the general white box to explore the internal problems of high-dimensional data theoretically and practically. Thus, we can reason to a proper assumption, which is also our further study, that is we want to find significant correlations of high-dimensional features across different machine learning models.

4.5 Top features and Heatmap

We also analyzed the top features returned by the model and plotted the importance of the top 30 features using Decision Tree on the dataset Entertainment + Social apps vs. Click Fraud + Adware malware. Through Figure 2, we can see that Opcode and API package features occupy the vast majority of top features. Opcode is usually the underlying code of machine instructions, and API packages are officially defined classes and libraries by Google Android API developer documentation.

For example, Opcode: add-long is the add binary operation of two long-type source registers, and Opcode: move-wide is the move operation of one register-pair's content to another. Also, the top feature API package: android.content is an ensemble of classes for accessing and publishing data on a device.

We also create a heatmap for analyzing the correlation between each top feature, and from the heatmap, we can find that the API package feature java.lang is independent of other Opcode features,

**Figure 2: Top 30 features of Entertainment + Social apps vs. Click Fraud + Adware dataset**

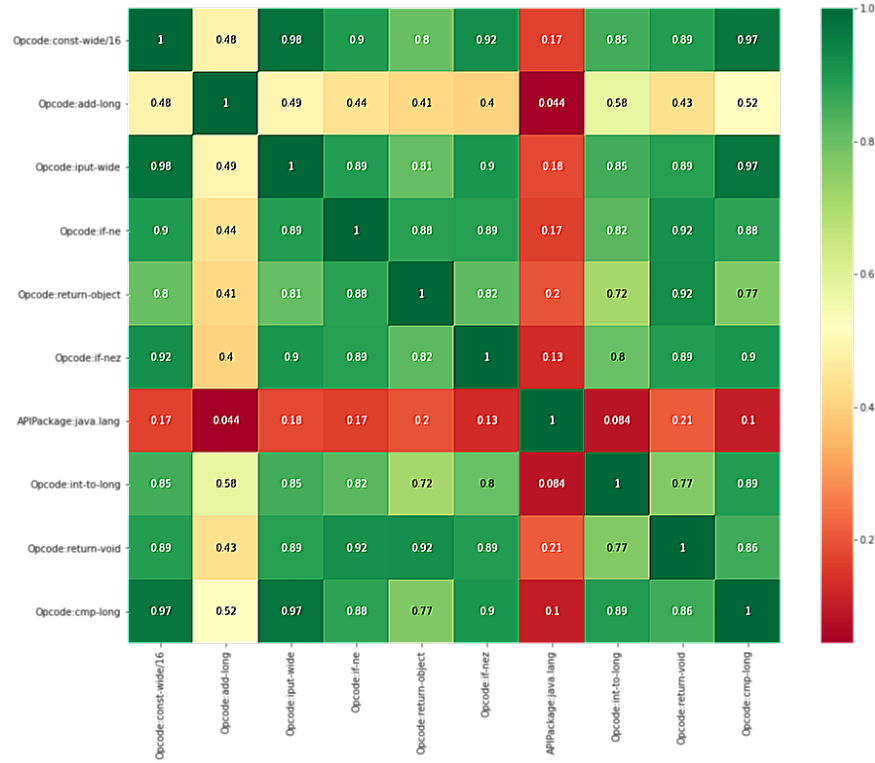


Figure 3: The heatmap of top 10 features' correlation coefficient of Entertainment + Social apps vs Click Fraud + Adware dataset

which all the correlation coefficient less than 0.21. However, for most Opcode top features, their correlation coefficients are generally higher than 0.7, which means that these opcodes usually positively correlate together to determine the actual behavior of the Android app.

5 CONCLUSION

In this paper, we proposed a new hybrid machine learning classification and anomaly detection method to detect current Android malware, with Precision, Recall, F1-Score, and AUC all stable at 0.997 and above. This method uses the idea of iteration of classification results and the essence of the anomaly algorithms to improve its stability in the experiment of imbalanced datasets. In addition, the anomaly detection algorithm in the fusion model accounts for approximately 50% and over of the whole, so the performance of the fusion model is better than that of the other four traditional machine learning models. More importantly, we added a data pre-processing process that was not present in the previous malware detection process. By categorizing goodware & malware, we are able to generate more insightful analysis and performance results in specific classified data that are more in line with the way processing is done with increasingly complex app detection. In the future, we will also consider applying the exception detection model proposed in this paper to the in-depth collection of different kinds of datasets, for achieving accurate detection of different malware scenarios in various environments. Meanwhile, we will also consider whether the feature dimensions under different datasets are interfered with

by traditional machine learning models and anomaly detection model algorithms so that we can improve its methods at the source of data collection. The above two explorations will make the practicability of the fusion model algorithm more extensive, rather than limited to the Android malware classification studied in this paper.

ACKNOWLEDGMENTS

We appreciate the anonymous reviewers for their efforts in reviewing our manuscripts. We additionally thank ViaX for excellent suggestions and comments on our manuscripts.

REFERENCES

- [1] Forecast number of mobile users worldwide from 2020 to 2025: <https://www.statista.com/statistics/218984/number-of-global-mobile-users-since-2010/>
- [2] Onwuzurike, L.; Mariconti, E.; Andriotis, P.; Cristofaro, E.D.; Ross, G.; Stringhini, G. MaMaDroid: Detecting Android malware by building Markov chains of behavioral models (extended version). *ACM Trans. Priv. Secur. (TOPS)* 2019, 22, 1–34.
- [3] Zhang, H.; Luo, S.; Zhang, Y.; Pan, L. An efficient Android malware detection system based on method-level behavioral semantic analysis. *IEEE Access* 2019, 7, 69246–69256.
- [4] Meng, G.; Xue, Y.; Xu, Z.; Liu, Y.; Zhang, J.; Narayanan, A. Semantic modelling of android malware for effective malware comprehension, detection, and classification. In *Proceedings of the 25th International Symposium on app Testing and Analysis*, Saarbrücken, Germany, 18–20 July 2016; pp. 306–317.
- [5] Mahindru, Arvind, and A. L. Sangal. "MLDroid—framework for Android malware detection using machine learning techniques." *Neural Computing and Applications* 33.10 (2021): 5183–5240.
- [6] Wang, Xinning, and Chong Li. "Android malware detection through machine learning on kernel task structures." *Neurocomputing* 435 (2021): 126–150.
- [7] Kim, Minki, et al. "Machine-Learning-Based Android Malware Family Classification Using Built-In and Custom Permissions." *Applied Sciences* 11.21 (2021):

- 10244.
- [8] Mahindru, Arvind, and A. L. Sangal. "SemiDroid: a behavioral malware detector based on unsupervised machine learning techniques using feature selection approaches." *International Journal of Machine Learning and Cybernetics* 12.5 (2021): 1369-1411.
- [9] Dam, Khanh Huu The, Thomas Given-Wilson, and Axel Legay. "Unsupervised behavioural mining and clustering for malware family identification." *Proceedings of the 36th Annual ACM Symposium on Applied Computing*. 2021.
- [10] Firdaus, Ahmad, *et al.* "Discovering optimal features using static analysis and a genetic search based method for Android malware detection." *Frontiers of Information Technology & Electronic Engineering* 19.6 (2018): 712-736.
- [11] Bakour, Khaled, H. Murat Ünver, and Razan Ghanem. "The android malware static analysis: techniques, limitations, and open challenges." *2018 3rd International Conference on Computer Science and Engineering (UBMK)*. Ieee, 2018.
- [12] Feng, Pengbin, *et al.* "A novel dynamic Android malware detection system with ensemble learning." *IEEE Access* 6 (2018): 30996-31011.
- [13] Sugunan, Krishna, T. Gireesh Kumar, and K. A. Dhanya. "Static and dynamic analysis for android malware detection." *Advances in Big Data and Cloud Computing*. Springer, Singapore, 2018. 147-155.
- [14] Kabakus, Abdullah Talha, and Ibrahim Alper Dogru. "An in-depth analysis of Android malware using hybrid techniques." *Digital Investigation* 24 (2018): 25-33.
- [15] Sun, Lichao, *et al.* "SigPID: significant permission identification for android malware detection." *2016 11th international conference on malicious and unwanted software (MALWARE)*. IEEE, 2016.
- [16] Chen, Tieming, *et al.* "TinyDroid: a lightweight and efficient model for Android malware detection and classification." *Mobile information systems 2018* (2018).
- [17] Nisa, Maryam, *et al.* "Hybrid malware classification method using segmentation-based fractal texture analysis and deep convolution neural network features." *Applied Sciences* 10.14 (2020): 4966.
- [18] Arp, Daniel, *et al.* "Drebin: Effective and explainable detection of android malware in your pocket." *Ndss*. Vol. 14. 2014.
- [19] Martín, Alejandro, Raul Lara-Cabrera, and David Camacho. "Android malware detection through hybrid features fusion and ensemble classifiers: the AndroPyTool framework and the OmniDroid dataset." *Information Fusion* 52 (2019): 128-142.
- [20] NJCCIC Android Malware Variants: <https://www.cyber.nj.gov/threat-center/threat-profiles/android-malware-variants/>
- [21] Lashkari, Arash Habibi, *et al.* "Toward developing a systematic approach to generate benchmark android malware datasets and classification." *2018 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 2018.
- [22] One-class SVM Novelty and Outlier Detection: https://scikit-learn.org/stable/modules/outlier_detection.html#outlier-detection